

Experiment 4

1. Aim

Implement process synchronization using Producer-Consumer Problem.

2. Hardware / Software Required

- Linux/Windows operating system
- GCC compiler
- C programming language
- Terminal/Command Prompt

3. Theory

The Producer-Consumer Problem is a classical process synchronization problem in Operating Systems.

- The Producer generates data and places it into a shared buffer.
- The Consumer removes and processes data from the buffer.
- Since both processes access the same buffer, synchronization is required to prevent conflicts.

A bounded buffer has a limited capacity. Therefore:

- The producer must wait when the buffer is full.
- The consumer must wait when the buffer is empty.
- Only one process should modify the shared buffer at a time.

Semaphores Used

Semaphore	Initial Value	Purpose
empty	Buffer size	Counts available empty spaces
full	0	Counts occupied spaces
mutex	1	Ensures mutual exclusion while accessing the buffer

Algorithm: Producer-Consumer Problem

Producer Algorithm

1. Start
2. Initialize the buffer and semaphores:
 - empty = BUFFER_SIZE
 - full = 0
 - mutex = 1
3. Generate an item.
4. Wait for an empty space using empty.
5. Lock the buffer using mutex.
6. Insert the item into the buffer.
7. Update the buffer insertion position.
8. Unlock the buffer.
9. Signal full to indicate that an item is available.
10. Repeat steps 3–9 until all items are produced.
11. Stop

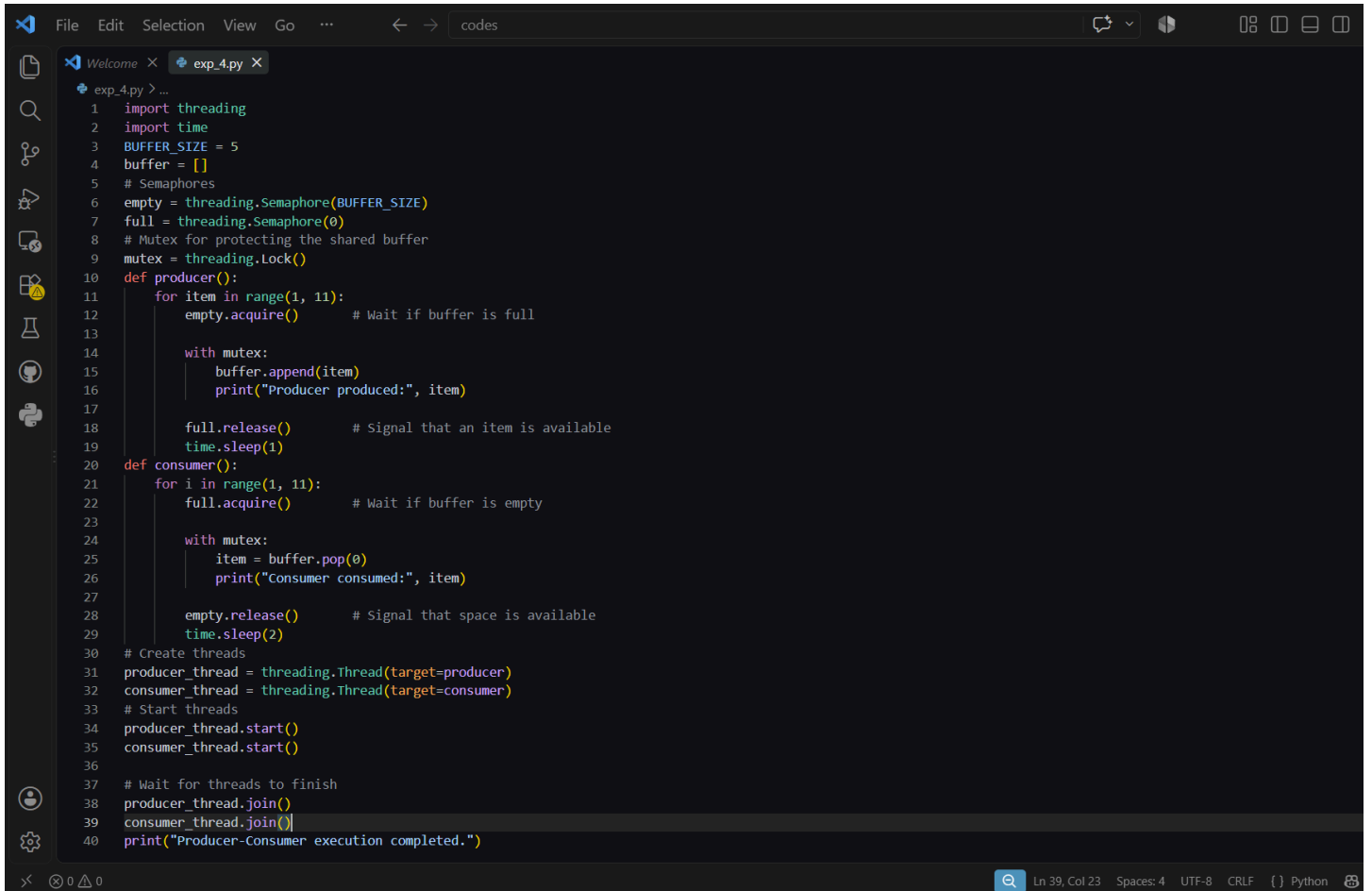
Consumer Algorithm

1. Start
2. Wait for an available item using full.
3. Lock the buffer using mutex.
4. Remove an item from the buffer.
5. Update the buffer removal position.
6. Unlock the buffer.
7. Signal empty to indicate that a space is available.
8. Process the consumed item.
9. Repeat steps 2–8 until all items are consumed.
10. Stop

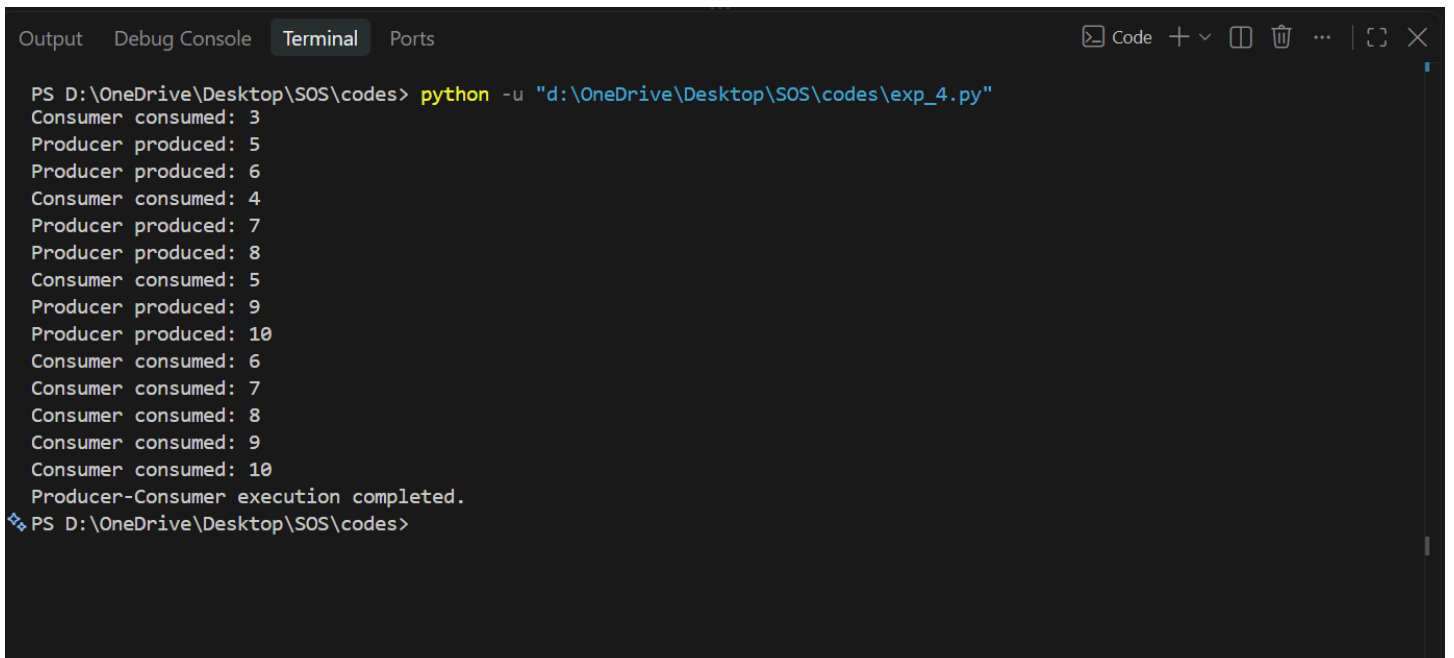
4. Procedure

1. Create a shared bounded buffer with a fixed capacity.
2. Create a producer process/thread to generate items.
3. Create a consumer process/thread to consume items.
4. Initialize the empty, full, and mutex semaphores.
5. The producer waits when the buffer is full.
6. The consumer waits when the buffer is empty.
7. Use the mutex semaphore to ensure only one process accesses the buffer at a time.
8. Insert and remove items from the shared buffer according to the semaphore conditions.
9. Compile and execute the program.
10. Observe the producer and consumer operations and verify that synchronization is maintained.

5. Output



```
File Edit Selection View Go ... codes
exp_4.py x
exp_4.py > ...
1 import threading
2 import time
3 BUFFER_SIZE = 5
4 buffer = []
5 # Semaphores
6 empty = threading.Semaphore(BUFFER_SIZE)
7 full = threading.Semaphore(0)
8 # Mutex for protecting the shared buffer
9 mutex = threading.Lock()
10 def producer():
11     for item in range(1, 11):
12         empty.acquire() # Wait if buffer is full
13
14         with mutex:
15             buffer.append(item)
16             print("Producer produced:", item)
17
18         full.release() # Signal that an item is available
19         time.sleep(1)
20 def consumer():
21     for i in range(1, 11):
22         full.acquire() # Wait if buffer is empty
23
24         with mutex:
25             item = buffer.pop(0)
26             print("Consumer consumed:", item)
27
28         empty.release() # Signal that space is available
29         time.sleep(2)
30 # Create threads
31 producer_thread = threading.Thread(target=producer)
32 consumer_thread = threading.Thread(target=consumer)
33 # Start threads
34 producer_thread.start()
35 consumer_thread.start()
36
37 # Wait for threads to finish
38 producer_thread.join()
39 consumer_thread.join()
40 print("Producer-Consumer execution completed.")
```



```
Output Debug Console Terminal Ports
Code + - [] [X] ... | [X] X
PS D:\OneDrive\Desktop\SOS\codes> python -u "d:\OneDrive\Desktop\SOS\codes\exp_4.py"
Consumer consumed: 3
Producer produced: 5
Producer produced: 6
Consumer consumed: 4
Producer produced: 7
Producer produced: 8
Consumer consumed: 5
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer consumed: 10
Producer-Consumer execution completed.
PS D:\OneDrive\Desktop\SOS\codes>
```

Conclusion

Thus, **process synchronization was successfully demonstrated using the Producer-Consumer Problem**. Semaphores were used to manage empty and occupied buffer spaces, while a mutex ensured mutual exclusion during buffer access. This prevents **race conditions and inconsistent data** when multiple processes or threads share a common resource.